

API SECURITY RISK ANALYSIS REPORT

Prepared By: NITHYASHREE
Internship: Future Interns
Track: Cyber Security (CS)
College: SRM IST Ramapuram

PROJECT OVERVIEW

This report presents an API Security Risk Analysis conducted as part of the Future Interns Cyber Security Internship Program. The objective of this assessment was to identify potential security risks and vulnerabilities in publicly accessible APIs.

Two widely used demo APIs, ReqRes API and JSONPlaceholder API, were selected for testing and evaluation. A total of seven security tests were performed to examine authentication mechanisms, input validation, data exposure, error handling, and access controls. The assessment resulted in eleven security findings, including four critical findings that could potentially impact the confidentiality, integrity, and availability of API services if left unaddressed.

API

- An API (Application Programming Interface) is a set of rules and protocols that allows different software applications to communicate and exchange data with each other. APIs act as intermediaries that enable applications to request and receive information or services without needing to understand the internal workings of the other system.
- For example, when a mobile application displays weather information, it uses an API to request data from a weather service provider. Similarly, online payment systems, social media platforms, and e-commerce websites use APIs to connect different services and share information securely.
- APIs play a crucial role in modern software development by enabling integration, automation, and efficient data exchange between systems, applications, and devices.

TOOLS AND METHODOLOGY

Tools Used:

ReqBin (reqbin.com)

- Used for sending API requests and testing API endpoints.
- Helped analyze API responses, request methods, and returned data.

Browser Developer Tools (DevTools)

- Used to inspect HTTP headers, network traffic, and API responses.
- Assisted in identifying security-related information exposed by APIs.

OWASP API Top 10

- Used as the primary security framework for identifying API vulnerabilities.
- Provided guidelines for evaluating authentication, authorization, data exposure, and other API security risks.

Methodology:

Step 1: Select Demo APIs

Publicly available demo APIs, including ReqRes API and JSONPlaceholder API, were chosen for security testing.

Step 2: Test API Endpoints

Various API endpoints were accessed using different HTTP requests to examine their behavior and functionality.

Step 3: Inspect Responses

API responses, headers, status codes, and returned data were analyzed for security weaknesses.

Step 4: Identify Security Risks

Potential vulnerabilities and misconfigurations were identified based on OWASP API security guidelines.

Step 5: Document Findings

All observations, risks, severity levels, and recommendations were recorded and organized into the final security assessment report.

WORKING MODEL

FINDING 1

Analysis

API: ReqRes

Status: 401 Unauthorized

Finding: API requires authentication/API key

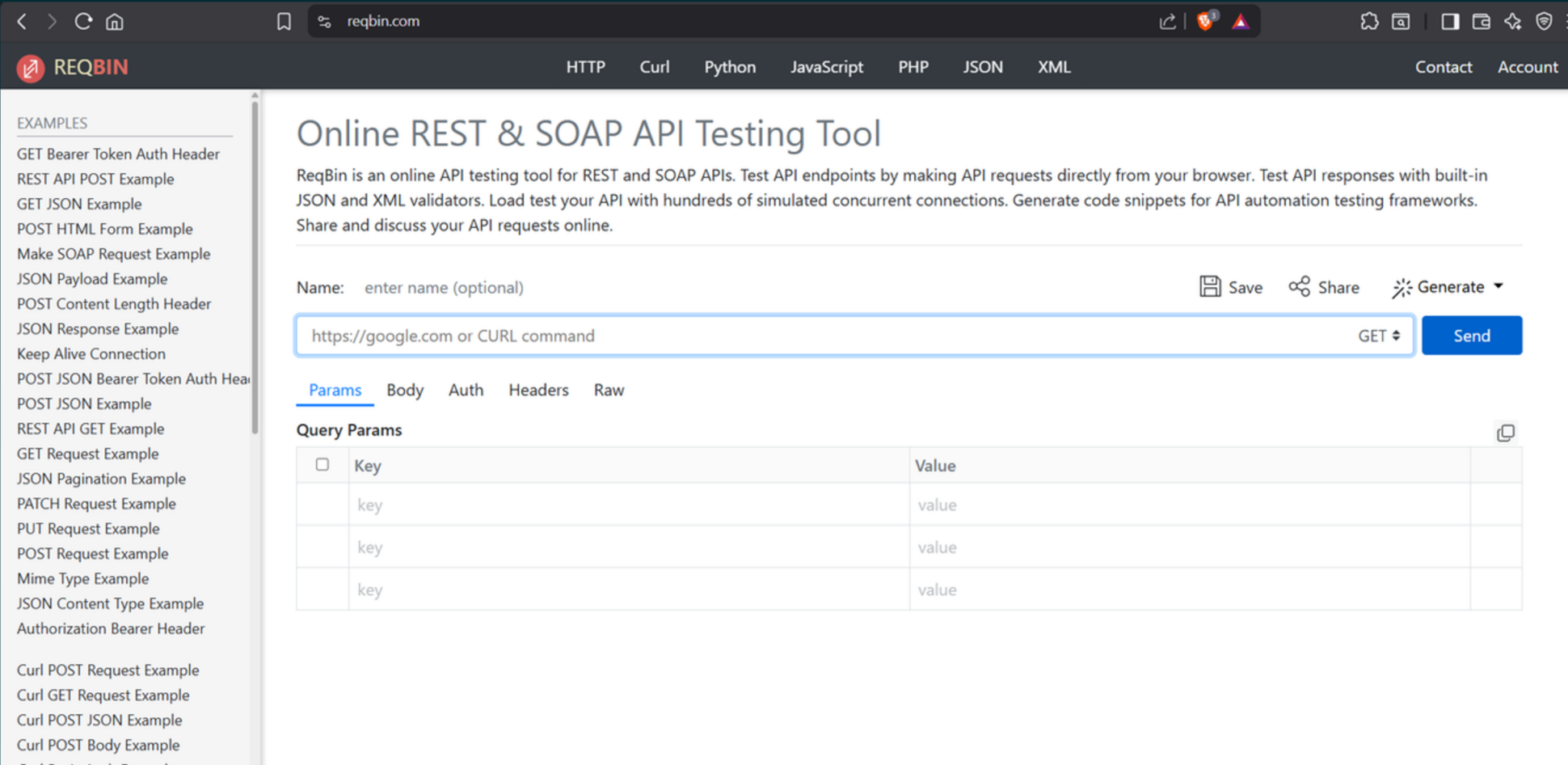
Risk Level: None – Good Security Practice

OWASP Category: Not Applicable (Secure Behavior)

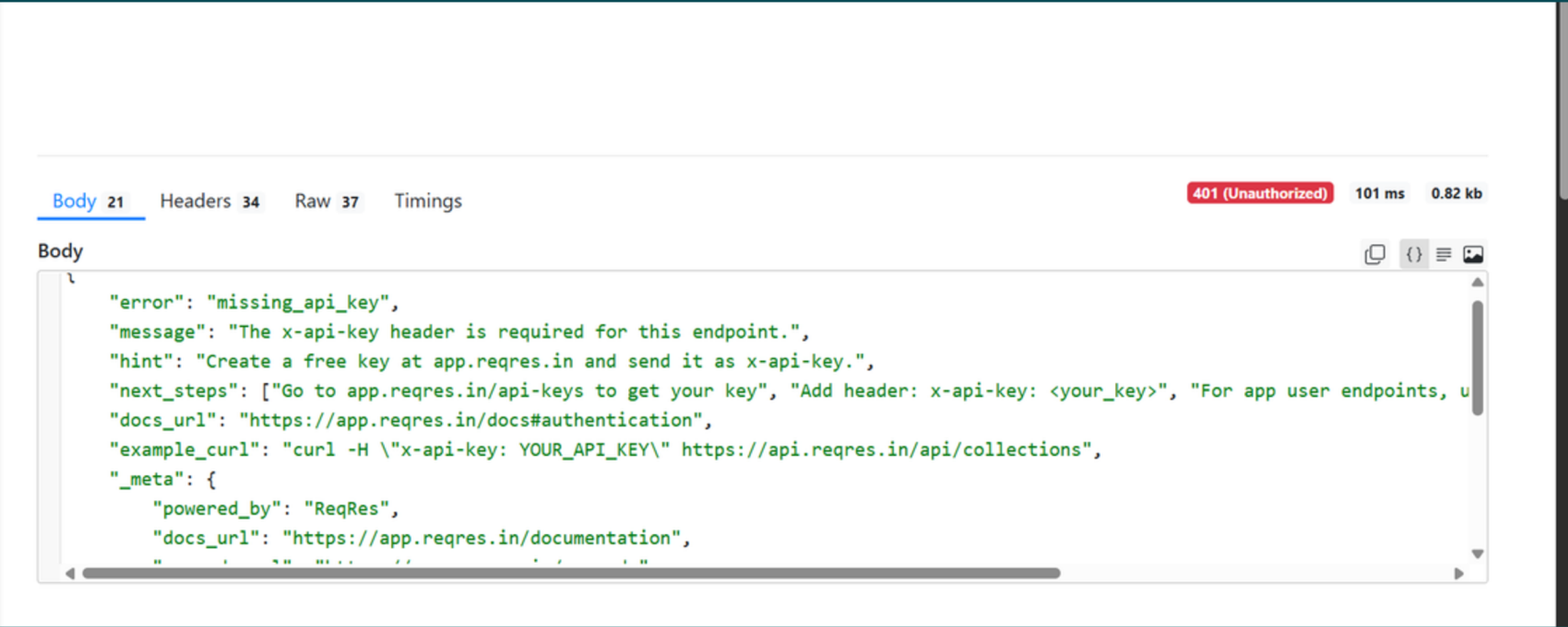
What It Means

During testing, a request was sent to the ReqRes API without providing valid authentication credentials. The API responded with a 401 Unauthorized status code, indicating that access was denied because the request lacked the required authorization.

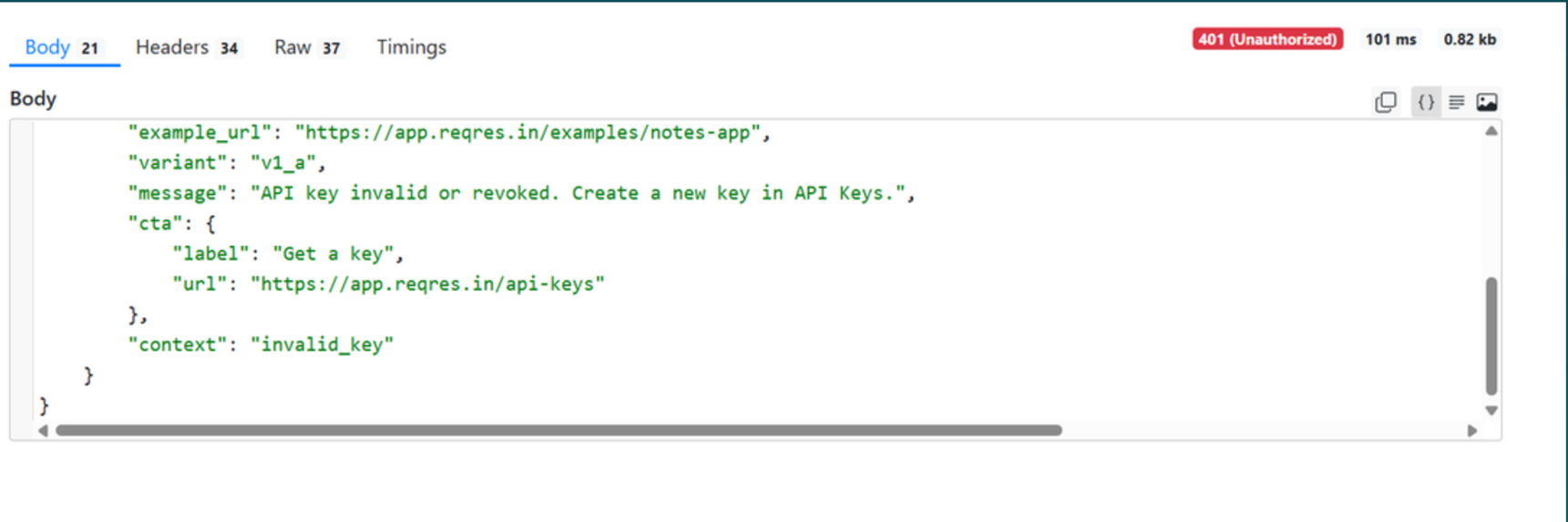
This is considered correct and secure behavior because the API prevents unauthorized users from accessing protected resources. By enforcing authentication requirements, ReqRes demonstrates good API security practices and reduces the risk of unauthorized access, data exposure, and misuse of API endpoints.



1



2



3

FINDING 2

Analysis

API: JSONPlaceholder

Status: 200 OK

Finding: Full user data exposed without authentication

Data Exposed

- User Names
- Email Addresses
- Physical Addresses
- Phone Numbers

Risk Assessment

Risk Level:  CRITICAL

OWASP Category: API1:2023 – Broken Object Level Authorization (BOLA)

What It Means

During testing, the JSONPlaceholder API returned user information successfully without requiring any authentication or authorization. The API responded with a 200 OK status code and exposed multiple user records, including names, email addresses, phone numbers, and address details.

In a real-world production environment, exposing such information without proper access controls could lead to privacy violations, data harvesting, targeted phishing attacks, and unauthorized access to sensitive user information. APIs should enforce authentication and authorization mechanisms before providing access to user-related data.

Online REST & SOAP API Testing Tool

ReqBin is an online API testing tool for REST and SOAP APIs. Test API endpoints by making API requests directly from your browser. Test API responses with built-in JSON and XML validators. Load test your API with hundreds of simulated concurrent connections. Generate code snippets for API automation testing frameworks. Share and discuss your API requests online.

Name:

 Save

 Share

 Generate 

GET 

Send

Params

Body

Auth

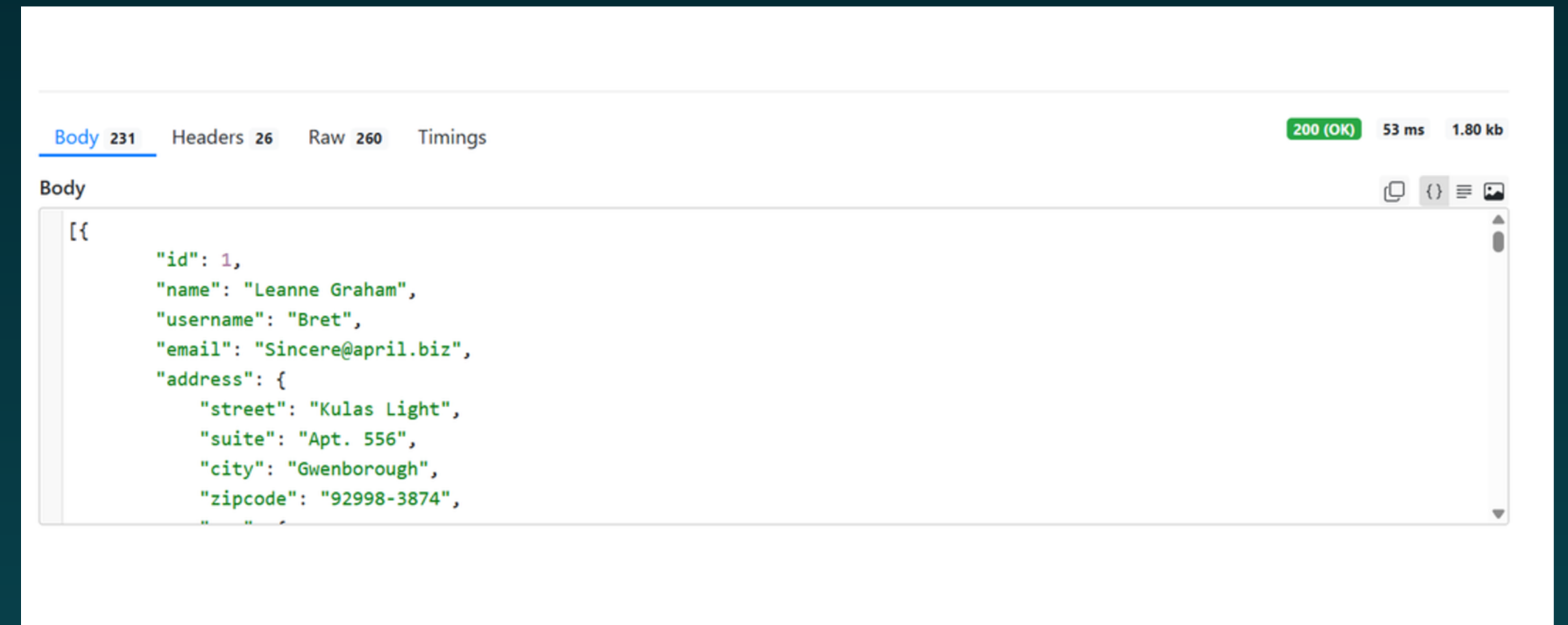
Headers 4

Raw 5

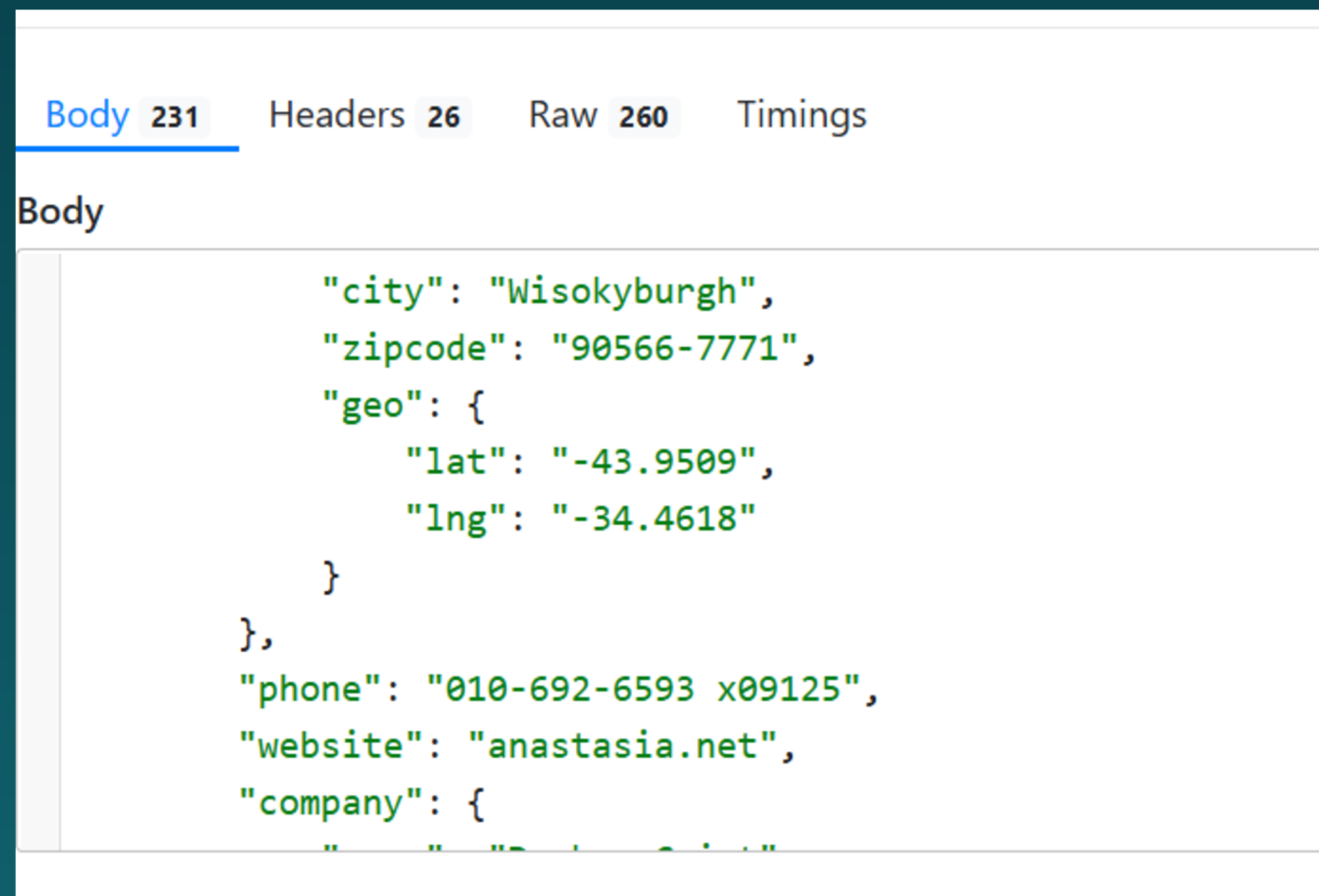
Query Params

☐	Key	Value	
☐	key	value	
☐	key	value	
☐	key	value	

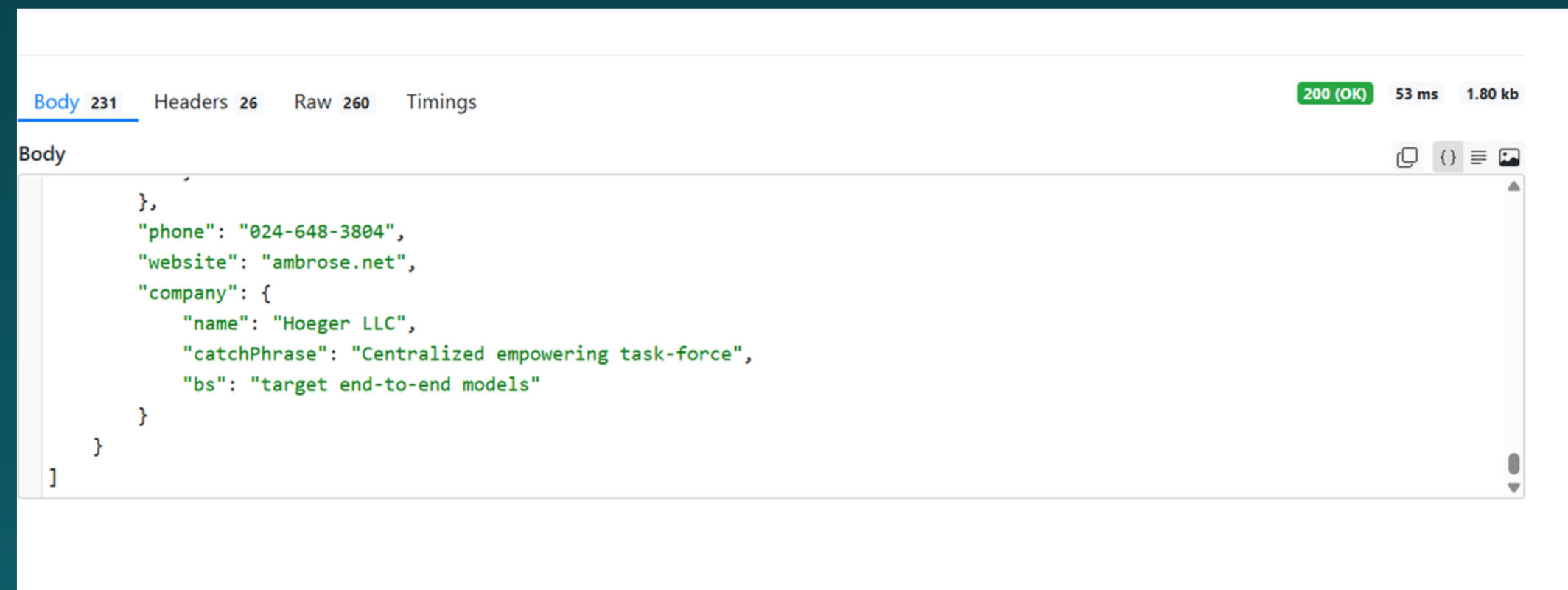
4



5



6



7

FINDING 3

Test Details

API: JSONPlaceholder

Request Method: POST

Endpoint: /users

Authentication: None Provided

Response Status: 201 Created

Finding:

The API accepted a POST request without requiring any authentication or authorization. The server successfully processed the request and returned a new record identifier, indicating that data creation was permitted without verifying the user's identity.

Security Issues Identified

- No authentication required
- No authorization checks performed
- Data creation allowed for anonymous users
- New record ID generated successfully
- Potential misuse of API resources

Risk Assessment

Risk Level:  Critical

OWASP Category: API2:2023 – Broken Authentication

Business Impact

If a production API behaves in this manner, attackers could create unauthorized accounts, submit fraudulent records, generate spam content, or abuse system resources without possessing valid credentials. Such weaknesses may also lead to unauthorized transactions, privilege abuse, and loss of data integrity.

Online REST & SOAP API Testing Tool

ReqBin is an online API testing tool for REST and SOAP APIs. Test API endpoints by making API requests directly from your browser. Test API responses with built-in JSON and XML validators. Load test your API with hundreds of simulated concurrent connections. Generate code snippets for API automation testing frameworks. Share and discuss your API requests online.

Name: enter name (optional)



Save



Share



Generate



jsonplaceholder.typicode.com/users

POST

Send

Params

Body

Auth

Headers 6

Raw 7

Query Params



☐	Key	Value	
☐	key	value	
☐	key	value	
☐	key	value	

8

Name: enter name (optional)



Save



Share



Generate



https://jsonplaceholder.typicode.com/users

POST

Send

Params

Body

Auth

Headers 6

Raw 7

☐ None

☒ JSON

☐ Form (url-encoded)

☐ XML

☐ Custom



```
{"key": "value"}
```

Body 3

Headers 25

Raw 30

Timings

201 (Created)

61 ms

0.01 kb

Body



```
{
  "id": 11
}
```

9

FINDING 4

Test Details

API: JSONPlaceholder
Input Sent:
<script>alert('xss')</script>
API Response: 201 Created

Finding

The API accepted and stored user-supplied script content without performing input validation or sanitization. The malicious script was successfully processed and included in the API response.

Risk Assessment

Risk Level:  Critical
OWASP Category: API8:2023 – Security Misconfiguration

What It Means

The API accepted potentially dangerous script input instead of filtering or rejecting it. In a real-world application, if such content is later displayed in a web page without proper encoding or sanitization, it could result in a Cross-Site Scripting (XSS) attack.

Business Impact

Attackers may inject malicious scripts that could steal user session information, capture sensitive data, redirect users to fraudulent websites, modify page content, or perform actions on behalf of victims. Such attacks can lead to data breaches, account compromise, and loss of user trust.

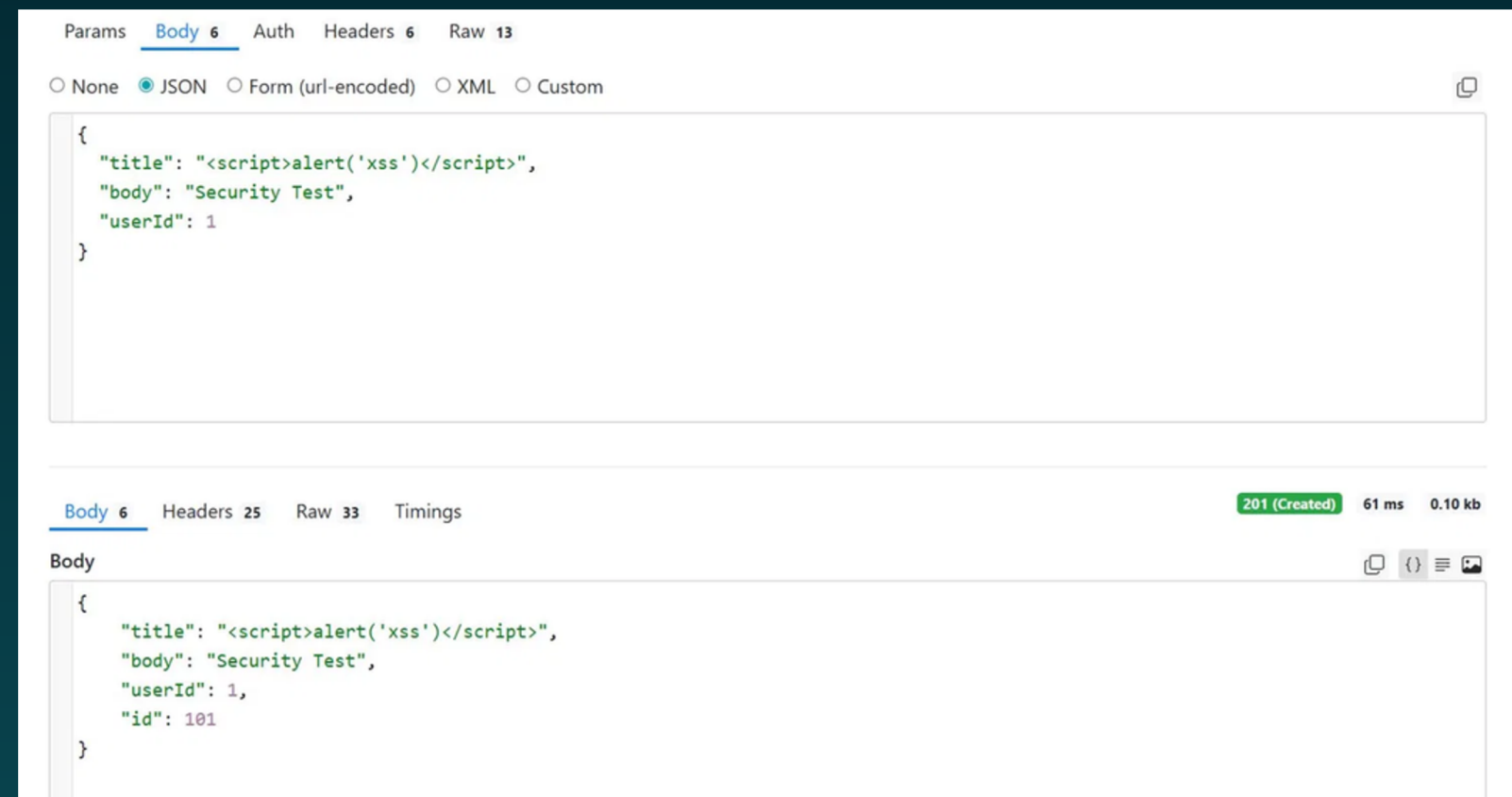


Fig10

- **Server accepted malicious script**
- **No input sanitization**
- **No validation at all**
- **Script stored in database**
- **201 Created – fully accepted!**

In a real application this means:

- **XSS attack possible**
- **Malicious scripts stored**
- **Can steal other users cookies**
- **Can redirect users to fake sites**
- **Complete account takeover!**

FINDING 5

Test Details

API: JSONPlaceholder

Method: DELETE

Response Status: 200 OK

Finding

The API accepted a DELETE request and processed it successfully without requiring any authentication or authorization. This indicates that data deletion actions can be performed without verifying the identity of the user.

Risk Assessment

Risk Level:  Critical

OWASP Category: API2:2023 – Broken Authentication

What It Means

During testing, a DELETE request was sent without providing login credentials, API keys, or authentication tokens. The API responded with a successful status code, demonstrating that deletion operations were allowed without proper access control.

Business Impact

In a real-world application, such a vulnerability could allow unauthorized users to delete important records, customer information, transactions, or business-critical data. Attackers could abuse this weakness to disrupt services, cause data loss, and negatively impact business operations.

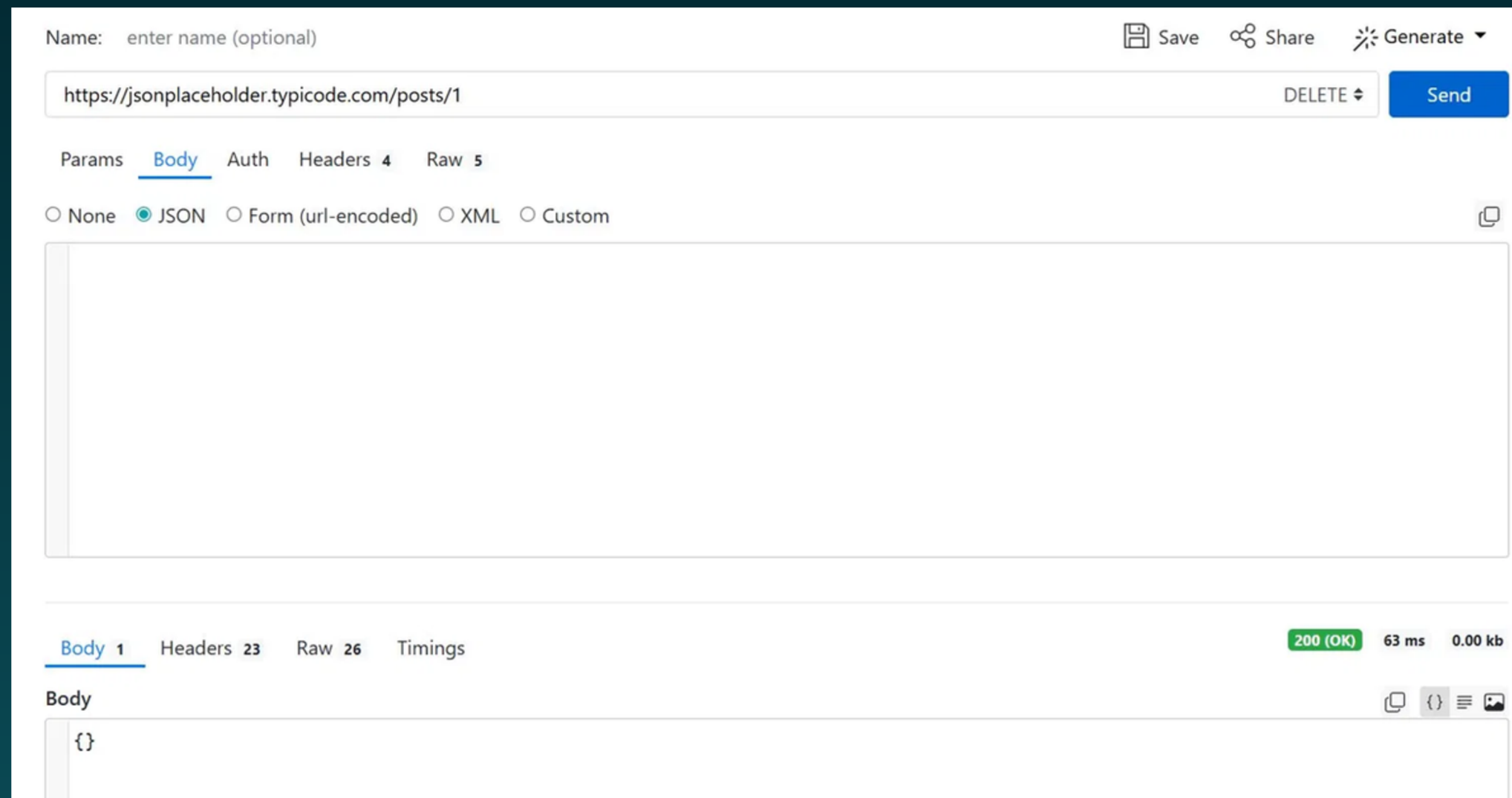


Fig11

In a real application this means:

- **Anyone can DELETE any data!**
- **No authentication needed**
- **No authorization check**
- **Entire database can be wiped!**
- **Competitors can delete your data**
- **Hackers can destroy everything!**

FINDING 6

Security Headers Analysis

Missing Security Headers

- Content-Security-Policy (CSP)
- X-Frame-Options
- Strict-Transport-Security (HSTS)
- Permissions-Policy

Additional Observation

-  x-powered-by: Express
- Reveals the underlying server technology.
- Provides information that attackers may use during reconnaissance activities.

Risk Assessment

- Risk Level:  Medium
- OWASP Category: API7:2023 – Security Misconfiguration
- What It Means

Security headers provide an additional layer of protection against common web-based attacks. During testing, several important security headers were found to be missing from the API responses. Additionally, the server disclosed its technology stack through the x-powered-by header.

Business Impact

The absence of security headers may increase exposure to attacks such as clickjacking, content injection, and protocol downgrade attacks. Server technology disclosure can also assist attackers in identifying potential vulnerabilities associated with the specific framework or software version being used.

Online REST & SOAP API Testing Tool

ReqBin is an online API testing tool for REST and SOAP APIs. Test API endpoints by making API requests directly from your browser. Test API responses with built-in JSON and XML validators. Load test your API with hundreds of simulated concurrent connections. Generate code snippets for API automation testing frameworks. Share and discuss your API requests online.

Name: enter name (optional)

Save

Share

Generate ▾

https://jsonplaceholder.typicode.com/users

GET ▾

Send

Params

Body

Auth

Headers 4

Raw 5

Headers

☐

Key

Value

key

value

key

value

key

value

Show All

12

Body	231	Headers	26	Raw	260	Timings	200 (OK)	36 ms
eaders								
		Cache-Control	max-age=43200					
		Content-Encoding	gzip					
		etag	W/"160d-1eMSsxeJRfnVLRBmYJSbCiJZ1qQ"					
		expires	-1					
		nel	{ "report_to": "heroku-nel", "response_headers": ["Via"], "max_age": 3600, "s...					
		pragma	no-cache					

14

Body	231	Headers	26	Raw	260	Timings	200 (OK)	36 ms	1.80 kb
Headers									
		Key	Value						
		Date	Tue, 02 Jun 2026 07:14:52 GMT						
		Content-Type	application/json; charset=utf-8						
		Content-Length	1847						
		Connection	keep-alive						
		access-control-allow-credentials	true						
		Cache-Control	max-age=43200						

13

	x-ratelimit-reset	1775756994
	Age	7057
	Accept-Ranges	bytes
	cf-cache-status	HIT
	CF-RAY	a054a9e808a1d954-EWR
	alt-svc	h3=":443"; ma=86400

15

PREVENTION

Enforce Strong Authentication

- **Require API keys, tokens, or OAuth authentication for all sensitive endpoints.**
- **Deny access to unauthenticated users.**

Implement Authorization Controls

- **Verify user permissions before allowing access, modification, or deletion of data.**
- **Apply Role-Based Access Control (RBAC).**

Protect Sensitive Data

- **Restrict access to personal and confidential information.**
- **Return only the data necessary for business operations.**

Validate User Input

- **Sanitize all user inputs to prevent malicious code injection.**
- **Reject invalid or suspicious requests.**

Secure Data Modification Operations

- **Require authentication for POST, PUT, PATCH, and DELETE requests.**
- **Log all changes for auditing purposes.**

Configure Security Headers

- **Enable Content-Security-Policy, X-Frame-Options, HSTS, and Permissions-Policy headers.**
- **Remove unnecessary server information from responses.**

Maintain Secure Configurations

- **Keep frameworks, libraries, and servers updated.**
- **Apply security patches promptly to reduce exposure to known vulnerabilities**

CONCLUSION

- **This API Security Risk Analysis was conducted to identify potential security weaknesses in publicly accessible APIs using industry-standard testing techniques and the OWASP API Security Top 10 framework. Two demo APIs, ReqRes and JSONPlaceholder, were assessed through multiple security tests focusing on authentication, authorization, data exposure, input validation, and security configuration.**
- **The assessment revealed that ReqRes demonstrated good security practices by enforcing authentication and preventing unauthorized access. In contrast, JSONPlaceholder exhibited several security weaknesses, including unauthorized data access, unrestricted data creation, acceptance of malicious input, unauthorized deletion operations, and missing security headers. These findings highlight the importance of implementing strong authentication, proper authorization controls, secure input validation, and robust security configurations.**
- **The results of this analysis emphasize that insecure APIs can expose sensitive information, enable unauthorized actions, and increase the risk of cyberattacks. Organizations should regularly assess their APIs, follow OWASP security recommendations, and implement preventive controls to protect data and maintain system integrity. Continuous monitoring, security testing, and employee awareness are essential for building secure and resilient API environments.**